18 Future Java

This chapter covers

- Project Amber
- Project Panama
- Project Loom
- Project Valhalla
- Java 18

This chapter covers developments in the Java language and platform since the release of Java 17, consisting of future updates that have not yet arrived. New directions in the Java language and platform are governed by JEPs, but those are descriptions of the implementations of specific features. At a higher level, there are several large, long-running projects within OpenJDK that are implementing the major changes that are currently in-flight and will be delivered over the coming years.

We're going to meet each project in turn and then Java 18. We're going to start with *Project Amber*, where we will hear more of the story of pattern matching and why it is such an important feature.

18.1 Project Amber

Of the current major projects in OpenJDK, Amber is the closest to completion. It also benefits from being relatively easy to understand in terms of a developer's day-to-day work. From the project's charter:

The goal of Project Amber is to explore and incubate smaller, productivity-oriented Java language features ...

—Project Amber, **https://openjdk.java.net/projects/amber/**

The main goals of the project are

- Local Variable Type Inference (Delivered)

- Switch Expressions (Delivered)
- Records (Delivered)
- Sealed Types (Delivered)
- Pattern Matching

As you can see, a lot of these features have been delivered as of Java 17—and very useful they are, too!

The last major piece of Amber that is still outstanding is Pattern Matching. As we saw in chapter 3, it is arriving in increments, the first of which is the use of a type pattern in `instanceof`. We also met the preview version of patterns in `switch`.

It is reasonable to expect that patterns in `switch` will move through the same lifecycle that other Project Amber features have: a first and then a second preview, and then delivery as a standard feature after that.

Looking to the future, more JEPs are planned. Finalizing the basic form of Pattern Matching is not the only game in town—there are additional forms of pattern to add. This tells us that, with the change in the release cadence to an "LTS every two years" model, anything initially previewed in Java 18 or 19 would have time to fully graduate for the next expected LTS, Java 21, in September 2023.

For example, we've already seen how Sealed Types can be used to great effect in the current, preview version of Pattern Matching. Without Sealed Types, even the form of Pattern Matching we have now would not be as useful. In a similar fashion, some of the most important use cases for Records in patterns have yet to be delivered. In particular, *deconstruction patterns* will allow a Record to be broken up in to its components as part of a pattern.

NOTE If you've programmed in Python or JS or other languages, you may be familiar with *destructuring*. The idea of deconstruction in Java is similar but is guided by Java's nominal type system.

This is possible because Records are defined by their semantics—a Record is literally nothing more than the sum of its parts. So, if a Record can be constructed only by stitching together its components, then it follows that it can be rendered down into its components with no semantic consequences.

At time of writing, this feature has not arrived into mainline JDK development, or even into the Amber-specific JDK repos. However, the syntax is expected to look like this:

```
FXOrder order = // ...

// WARNING This is preliminary syntax!!!

var isMarket = switch (order) {
   case MarketOrder(int units, CurrencyPair pair, Side side,
                    LocalDateTime sent, boolean allOrNothing) -> true;
   case LimitOrder(int units, CurrencyPair pair, Side side,
                   LocalDateTime sent double price, int ttl) -> false;
};
```

Note that this code contains explicit types for the Record components. It is also reasonable to expect that the types could also be inferred by the compiler.

It should also be possible to deconstruct arrays as well because they also act as element containers that do not have additional semantics. The syntax for that may look like this:

```
// WARNING This is preliminary syntax!!!

if (o instanceof String[] { String s1, String s2, ... }){
   System.out.println(s1 + s2);
}
```

Note that in both examples, we have not declared a binding for the element container, whether a Record or an array.

A side point that should be mentioned here is how Java serialization affects this feature. In general, Java serialization is a problem, because it violates some basic rules of how encapsulation is supposed to work in Java.

Serialization constitutes an invisible but public constructor, and an invisible but public set of accessors for your internal state.

—Brian Goetz

Fortunately, both Records and arrays are very simple: they are just transparent carriers for their contents, so there is no need to invoke the weirdness in the detail of the serialization mechanism. Instead, we can always use the public API and canonical constructor to serialize and deserialize records. Building upon this foundation, there are even suggestions that could be very far-reaching, such as removing the serialization mechanism partially or completely and extending deconstruction to some (or even all) Java classes.

Overall, the message from Amber is: if you're familiar with these features from other programming languages, then great. But, if not, then don't worry—they are being designed to fit with the Java language you already know and be easy to start using in your code.

Although some of the features are small and others larger, they can all have a positive impact on your code that is out of proportion with the size of the changes. Once you've started using them, you'll likely find that they offer real benefit to your programs. Let's now turn to the next of the major projects, codenamed Panama.

18.2 Project Panama

Project Panama is, in the words of its project page, all about

improving and enriching the connections between the Java virtual machine and well-defined but "foreign" (non-Java) APIs, including many interfaces commonly used by C programmers.

—Project Panama, **https://openjdk.org/projects/panama/**

The name "Panama" comes from the idea of an *isthmus*—a narrow string of land connecting two larger landmasses—which in this analogy are understood to be the JVM and native code. It comprises JEPs in two main areas:

- Foreign Function and Memory API
- Vector API

Of these, we will discuss only the Foreign API in this section. The Vector API is not ready for a full treatment yet, for reasons that we will explain later in the chapter.

3.2.1 Foreign Function and Memory API

Java has had the Java Native Interface (JNI) for calling in to native code since Java 1.1, but it has long been recognized as having the following major problems:

- The JNI has a lot of ceremony and extra artifacts.
- The JNI really only interoperates well with libraries written in C and C++.
- The JNI does not do anything automatic to map the Java type system to the C type system.

The extra artifacts aspect is reasonably well understood by developers: as well as the Java API of `native` methods, JNI requires a C header (`.h`) file derived from the Java API

and a C implementation file, which will call into the native library. Some of the other aspects are less well known, such as the fact that a native method cannot be used to invoke a function written in a language that uses a different calling convention than the one the JVM was built against.

In the intervening years since the JNI first arrived, a number of attempts have been made to provide a better alternative, such as JNA. However, other (non-JVM) languages have significantly better support for interoperating with native code. For example, Python's reputation as a good language for machine learning largely depends on the ease of packaging native libraries and making them available in Python code.

The Panama Foreign API is an attempt to close that gap, by allowing direct support in Java for the following:

- Foreign memory allocation
- Manipulation of structured foreign memory
- Lifecycle management of foreign resources
- Calling foreign functions

The API lives in the `jdk.incubator.foreign` package in the `jdk.incubator.foreign` module. It builds upon MethodHandles and VarHandles, which we met in chapter 17.

NOTE The Foreign API is contained in an incubator module in Java 17. We discussed incubator modules and their significance way back in chapter 1. To get the code examples in this section to run, you will need to explicitly add the incubator module to your modulepath.

The first piece of the API relies on classes like `MemorySegment`, `MemoryAddress`, and `SegmentAllocator`. This provides access to allocation and handling of off-heap memory. The aim is to provide a better alternative to the use of both the ByteBuffer API and `Unsafe`. The Foreign API intends to avoid the limitations of `ByteBuffer`, such as performance, being limited to segments 2 GB in size and being not specifically designed for off-heap use. At the same time, it should be, well, safer than the use of `Unsafe`, which allows basically unrestricted memory access, making it very easy for bugs to crash the JVM.

NOTE For the rest of this section, we assume that you are familiar with C language concepts, as well as building C/C++ programs from source and understand the phases of C compilation, linking, and so on.

Let's see it in action. To get started, you will need to download an early-access build of Panama from **https://jdk.java.net/panama/**. Although the incubator modules are present in JDK 17, the important `jextract` tool is not, and we'll need it for our example.

Once you have a Panama early-access install set up, test it with `jextract -h`. You should see output like this:

```
WARNING: Using incubator modules:
      jdk.incubator.jextract, jdk.incubator.foreign
Non-option arguments:
[String] -- header file

Option                  Description
------                  -----------
-?, -h, --help            print help
-C <String>               pass through argument for clang
-I <String>               specify include files path
-d <String>               specify where to place generated files
--dump-includes <String>      dump included symbols into specified file
--header-class-name <String>   name of the header class
--include-function <String>    name of function to include
--include-macro <String>      name of constant macro to include
--include-struct <String>     name of struct definition to include
--include-typedef <String>    name of type definition to include
--include-union <String>      name of union definition to include
--include-var <String>        name of global variable to include
-l <String>               specify a library
--source                  generate java sources
-t, --target-package <String>  target package for specified header files
```

For our example, we're going to use a simple PNG library that's written in C: LibSPNG (**https://libspng.org/**).

EXAMPLE: LIBSPNG

We're going to start by using the `jextract` tool to get a set of base Java packages that we can use. The syntax looks like this:

```
$ jextract --source -t <target Java package> -l <library name> \
   -I <path to /usr/include> <path to header file>
```

On a Mac, this ends up being something like this:

```
$ jextract --source -t org.libspng \
  -I /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/ \
     Developer/SDKs/MacOSX.sdk/usr/include \
  -l spng /Users/ben/projects/libspng/spng/spng.h
```

This may give some warnings, depending on exactly the version of the header file we're generating from, but providing it succeeds, it will create a directory structure in the current directory. This will contain a lot of Java classes in a package called `org.libspng`, which we'll be able to use from within our Java program later.

We also need to build a shared object to link against when we run our program. This is best accomplished following the project's build instructions at **http://mng.bz/v6dJ**.

The installation generates `libspng.dylib` locally within the project and installs it to a system-shared location. When running the project, you'll want to ensure that the file is somewhere in the paths listed by the system property `java.library.path`, or directly set that property to include your location. An example default directory on a Mac is `~/Library/Java/Extensions/`. With the code generation completed and the library installed, we can get on with some Java programming.

The aim of Panama is to provide Java static methods that match the names (and Java versions of the parameter types) of the symbols in the C library that we want to link against. So, the symbols in the generated Java code will follow C naming conventions and won't look much like Java names.

For the Java programmer, the overall impression is that we're calling the C functions directly (as far as possible). In reality, there's a certain amount of Panama magic that is happening under the hood, using techniques like method handles to hide the complexity. Under normal circumstances, most developers need not worry about the details of exactly how Panama works.

Let's take a look at an example: a program that uses a C library to read some basic data from a PNG file. We'll set up this code as a proper modular build. The module descriptor, `module-info.java`, looks like this:

```
module wgjd.png {
  exports wgjd.png;

  requires jdk.incubator.foreign;
}
```

The code comprises the packages `org.libspng`, which we autogenerated from the C code, and the single exported package, `wgjd.png`. It contains a single file, which we're showing in full as the imports and so on are important to understand what's happening:

```
package wgjd.png;

import jdk.incubator.foreign.MemoryAddress;                    (1)
import jdk.incubator.foreign.MemorySegment;                    (1)
import jdk.incubator.foreign.SegmentAllocator;                 (1)
import org.libspng.spng_ihdr;

import static jdk.incubator.foreign.CLinker.toCString;
import static jdk.incubator.foreign.ResourceScope.newConfinedScope;
import static org.libspng.spng_h.*;

public class PngReader {
    public static void main(String[] args) {
        if (args.length < 1) {
            System.err.println("Usage: pngreader <fname>");
            System.exit(1);
        }

        try (var scope = newConfinedScope()) {
            var allocator = SegmentAllocator.ofScope(scope);

            MemoryAddress ctx = spng_ctx_new(0);               (2)
            MemorySegment ihdr = allocator.allocate(spng_ihdr.$LAYOUT());

            spng_set_crc_action(ctx, SPNG_CRC_USE(),           (3)
                                SPNG_CRC_USE());

            int limit = 1024 * 1024 * 64;                      (4)
            spng_set_chunk_limits(ctx, limit, limit);          (2)

            var cFname = toCString(args[0], scope);            (5)
            var cMode = toCString("rb", scope);                (5)
            var png = fopen(cFname, cMode);                    (6)
            spng_set_png_file(ctx, png);                       (2)

            int ret = spng_get_ihdr(ctx, ihdr);                (2)

            if (ret != 0) {
                System.out.println("spng_get_ihdr() error: " +
                                   spng_strerror(ret));
                System.exit(2);
```

```
        }

        final String colorTypeMsg;
        final byte colorType = spng_ihdr.color_type$get(ihdr);

        if (colorType ==
            SPNG_COLOR_TYPE_GRAYSCALE()) {                    ❸
          colorTypeMsg = "grayscale";
        } else if (colorType ==
            SPNG_COLOR_TYPE_TRUECOLOR()) {                    ❼
          colorTypeMsg = "truecolor";
        } else if (colorType ==
            SPNG_COLOR_TYPE_INDEXED()) {                      ❼
          colorTypeMsg = "indexed color";
        } else if (colorType ==
            SPNG_COLOR_TYPE_GRAYSCALE_ALPHA()) {              ❼
          colorTypeMsg = "grayscale with alpha";
        } else {
          colorTypeMsg = "truecolor with alpha";
        }

        System.out.println("File type: " + colorTypeMsg);
      }
    }
}
```

❶ Panama classes for working with C-style memory management

❷ The Java wrapper for a C function in spng.h

❸ The Java wrapper for a C constant

❹ Reads data in 64 M chunks

❺ Copies contents of the Java string into a C string

❻ The Java wrapper for a C standard library function

❼ The Java wrapper for a C constant

This is built with a Gradle build script, shown here:

```
plugins {
  id("org.beryx.jlink") version("2.24.2")
```

```
}

repositories {
 mavenCentral()
}

application {
 mainModule.set("wgjd.png")
 mainClass.set("wgjd.png.PngReader")
}

java {
   modularity.inferModulePath.set(true)
}

sourceSets {
 main {
  java {
   setSrcDirs(listOf("src/main/java/org",
                "src/main/java/wgjd.png"))
  }
 }
}

tasks.withType<JavaCompile> {
 options.compilerArgs = listOf()
}

tasks.jar {
 manifest {
  attributes("Main-Class" to application.mainClassName)
 }
}
```

And can be executed like this:

```
$ java --add-modules jdk.incubator.foreign \
   --enable-native-access=ALL-UNNAMED \
   -jar build/libs/Panama.jar <FILENAME>.png
```

This should produce some output providing basic metadata about our image file.

One key aspect of handling memory is the question of the lifetime of native memory. C does not have a garbage collector, so all memory must be manually allocated and de-allocated. This is, of course, extremely error prone as well as being not at all natural for a Java programmer.

To work around this problem, Panama provides several classes that are used as Java handles for C memory management operations. The key is the `ResourceScope` class, which can be used to provide deterministic cleanup. This is handled in the usual Java way, via `try` -with-resources. For example, the previous code used a lexically scoped lifetime for the native memory handling:

```
try (var scope = newConfinedScope()) {
    var allocator = SegmentAllocator.ofScope(scope);

    // ...

}
```

The `allocator` object is an instance of an implementation of the `SegmentAllocator` interface. It is created from the scope via a factory method, and in turn we can create `MemorySegment` objects from the allocator.

Objects that implement the `MemorySegment` interface represent contiguous blocks of memory. Typically, these will be backed by blocks of native memory, but it is also possible to back memory segments with on-heap arrays. This is similar to the case of `ByteBuffer` in the Java NIO API.

NOTE The Panama API also contains `MemoryAddress`, which is effectively a Java wrapper over a C pointer (expressed as a `long` value).

When the scope is autoclosed, the allocator will be called back to deterministically deallocate and release any resources that it had been holding. This is how the *Resource Acquisition Is Initialization* (or RAII) pattern, which is implemented in Java using `try` -with-resources, is carried into native code. The scope and allocator objects hold references to native resources and automatically free them when the TWR block exits.

Alternatively, this can be handled implicitly, with the native memory being cleaned up when a `MemorySegment` object is garbage-collected. This, of course, means that the

cleanup happens nondeterministically, whenever the GC runs. In general, it is advisable to use explicit scopes, especially if you are not familiar with the potential pitfalls of handling off-heap memory.

At time of writing, `jextract` understands only C header files. This means that, at present, to use it from other native languages (e.g., Rust), you have to generate a C header first. Ideally, there would be an automatic tool to generate these, which would work like the `rust-bindgen` tool but in reverse.

More broadly, over time, `jextract` may get more support for other languages generally. The tool is based on LLVM, which is already language independent, so, theoretically, it should be extensible to any language LLVM knows and which can handle C function call conventions.

The Foreign API is about to see its second release as an Incubating feature (see way back in chapter 1 for the description of Incubating and Preview features) as part of Java 18. It is hoped that it will become a final, standardized feature as part of Java 19 in September 2022.

The Vector API is not as advanced, primarily because the API designers have decided that they would prefer to wait until the capabilities of Project Valhalla (see later in this chapter) are available. This API therefore will not move out of Incubating status until Valhalla is available as a standard feature.

18.3 Project Loom

In its own words, OpenJDK's *Project Loom* is about

easy-to-use, high-throughput lightweight concurrency and new programming models on the Java platform.

—Project Loom, **https://wiki.openjdk.org/display/loom/Main**

Why is this new approach to concurrency needed? Let's consider Java from a more historical perspective.

One interesting way of thinking about Java is that it is a late-1990s language and platform that made a number of opinionated, strategic bets about the direction of evolution of software. Those bets, from the perspective of 2022, have largely paid off (whether more by luck or by judgment is a matter for debate, of course).

As an example, consider threading. Java was the first mainstream programming platform to bake threads into the core language. Before threads, the state of the art was to use multiple processes and various unsatisfactory mechanisms (Unix shared memory, anyone?) to communicate between them.

At an operating system level, threads are independently scheduled execution units that belong to a process. Each thread has an execution instruction counter and a call stack, but shares a heap with every other thread in the same process.

Not only that, but the Java heap is just a single contiguous subset of the process heap (at least in the HotSpot implementation—other JVMs may differ), so the memory model of threads at an OS level carries over naturally to the Java language domain.

The concept of threads naturally leads to a notion of a lightweight context switch. It is cheaper to switch between two threads in the same process than otherwise. This is primarily because the mapping tables that convert from virtual memory addresses to physical ones are mostly the same for threads in the same process.

NOTE Creating a thread is also cheaper than creating a process. The exact extent to which this is true depends on the details of the operating system in question.

In our case, the Java specification does not mandate any particular mapping between Java threads and operating system (OS) threads (assuming that the host OS even has a suitable thread concept, which has not always been the case). In fact, in very early Java versions, the JVM threads were multiplexed onto OS (aka *platform*) threads in what were referred to as *green threads* or *M:1 threads* (because the implementation actually used only a single platform thread).

However, this practice died away around the Java 1.2/1.3 era (and slightly earlier on the Sun Solaris OS), and modern Java versions running on mainstream operating systems instead implement the rule that one Java thread == exactly one operating system thread. Calling `Thread.start()` calls the thread creation system call (e.g., `clone()` on Linux) and actually creates a new OS thread.

OpenJDK's Project Loom's primary goal is to enable new `Thread` objects that can execute code but do not correspond to dedicated OS threads, or, to put it another way, to create an execution model where an object that represents an execution context is not necessarily a thing that needs to be scheduled by the operating system.

So in some respects, Loom is a return to something similar to green threads. However, the world has changed a lot in the intervening years, and sometimes in computing, there

are ideas that are ahead of their time.

For example, one could regard Enterprise Java Beans (EJBs) as a form of virtualized/ restricted environment that overambitiously tried to virtualize the environment away. Can they perhaps be thought of as a prototypical form of the ideas that would later find favor in modern PaaS systems—and to a lesser extent in Docker/K8s?

So, if Loom is a (partial) return to the idea of green threads, then one way of approaching it might be via the query: "what has changed in the environment that makes it interesting to return to an old idea that was not found to be useful in the past?"

To explore this question a little, let's look at an example. Specifically, let's try to crash the JVM by creating too many threads. You should not run the code in this example unless you are prepared for a possible crash:

```
//
// Do not actually run this code... it may crash your JVM or laptop
//
public class CrashTheVM {
    private static void looper(int count) {
        var tid = Thread.currentThread().getId();
        if (count > 500) {
            return;
        }
        try {
            Thread.sleep(10);
            if (count % 100 == 0) {
                System.out.println("Thread id: "+ tid +" : "+ count);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        looper(count + 1);
    }

    public static Thread makeThread(Runnable r) {
        return new Thread(r);
    }

    public static void main(String[] args) {
        var threads = new ArrayList<Thread>();
        for (int i = 0; i < 20_000; i = i + 1) {
            var t = makeThread(() -> looper(1));
            t.start();
```

```
            threads.add(t);
            if (i % 1_000 == 0) {
                System.out.println(i + " thread started");
            }
        }
        // Join all the threads
        threads.forEach(t -> {
            try {
                t.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
    }
}
```

The code starts up 20,000 threads and does a minimal amount of processing in each one—or tries to. In practice, it will often die or lock up the machine long before that steady state is reached.

NOTE It is possible to get the example to run through to completion if the machine or OS are throttled and can't create threads fast enough to induce the resource starvation.

Although it is obviously not completely representative, this example is intended to signpost what will happen to, for example, a web serving environment with one thread per connection. It is entirely reasonable for a modern high-performance web server to be expected to handle 20,000 concurrent connections, and yet this example clearly demonstrates the failure of a thread-per-connection architecture for that case.

NOTE Another way to think about Loom is that a modern Java program may need to keep track of many more executable contexts than it can create threads for.

An alternative takeaway could be that threads are potentially much more expensive than we think and represent a scaling bottleneck for modern JVM applications. Developers have been trying to solve this problem for years, either by taming the cost of threads or by using a representation of execution contexts that aren't threads.

One way of trying to achieve this was the SEDA approach (Staged Event Driven Architecture)—roughly speaking, a system in which a domain object is moved from A to Z along a multistage pipeline with various different transformations happening along the way. This can be implemented in a distributed system using a messaging system or in a single process, using blocking queues and a thread pool for each stage.

At each step, the processing of the domain object is described by a Java object that contains code to implement the step transformation. For this to work correctly, the code must be guaranteed to terminate—no infinite loops—and this cannot be enforced by the framework.

This approach has some notable shortcomings—not least the discipline required by programmers to use the architecture effectively. Let's take a look at a better alternative.

3.3.1 Virtual threads

Project Loom aims to deliver a better experience for today's high-scale applications by adding the following new constructs to the JVM:

- Virtual threads
- Delimited continuations
- Tail-call elimination

The key aspect of this is *virtual threads*. These are designed to look to the programmer like "just threads." However, they are managed by the Java runtime and are *not* thin, one-to-one wrappers over OS threads. Instead, they are implemented in user space by the Java runtime. The major advantages that virtual threads are intended to bring include

- Creating and blocking them is cheap.
- Standard Java execution schedulers (threadpools) can be used.
- No OS-level data structures are needed for the stack.

The removal of the involvement of the operating system in the lifecycle of a virtual thread is what removes the scalability bottleneck. Our JVM applications can cope with having millions or even billions of objects—so why are we restricted to just a few thousand OS-schedulable objects (which is one way to think about what a thread is)? Shattering this limitation and unlocking new concurrent programming styles is the main aim of Project Loom.

Let's see virtual threads in action. Download a Loom beta (**https://jdk.java.net/loom/**), and spin up `jshell` (with Preview mode enabled to activate the Loom features) like this:

```
$ jshell --enable-preview
|  Welcome to JShell -- Version 18-loom
|  For an introduction type: /help intro
```

```
jshell> Thread.startVirtualThread(() -> {
   ...>     System.out.println("Hello World");
   ...> });
Hello World
$1 ==> VirtualThread[<unnamed>,<no carrier thread>]

jshell>
```

We can straightaway see the virtual thread construct in the output. We are also using a new static method `startVirtualThread()` to start the lambda in a new execution context, which is a virtual thread. Simple!

The general rule has to be that existing codebases must continue to run in exactly the way that they did until the advent of Loom. Or, to put it another way, using virtual threads must be opt-in. We must make the conservative assumption that all existing Java code genuinely needs the lightweight wrapper over OS threads that has been, until now, the only game in town.

The arrival of virtual threads opens up new horizons in other ways. Until now, the Java language has offered the following two primary ways of creating new threads:

- Subclass `java.lang.Thread`, and call the inherited `start()` method.
- Create an instance of `Runnable`, and pass it to a `Thread` constructor, then start the resulting object.

If the notion of what a thread *is* will be changing, then it makes sense to re-examine the methods we use to create threads as well. We have already met the new static factory method for *fire-and-forget* virtual threads, but the existing thread API needs to be improved in a few other ways as well.

3.3.2 Thread builders

One important new notion is the `Thread.Builder` class, which has been added as an inner class of `Thread`. Two new factory methods have been added to `Thread` to give access to builders for platform and virtual threads, shown here:

```
jshell> var tb = Thread.ofPlatform();
tb ==> java.lang.ThreadBuilders$PlatformThreadBuilder@312b1dae

jshell> var tb = Thread.ofVirtual();
tb ==> java.lang.ThreadBuilders$VirtualThreadBuilder@506e1b77
```

Let's see the builder in action by replacing the `makeThread()` method in our example with this code:

```
// Loom-specific code
public static Thread makeThread(Runnable r) {
   return Thread.ofVirtual().unstarted(r);
}
```

This calls the `ofVirtual()` method to explicitly create a virtual thread that will execute our `Runnable`. We could, of course, have used the `ofPlatform()` factory method instead, and we would have ended up with a traditional, OS-schedulable thread object. But where's the fun in that?

If we substitute the virtual version of `makeThread()` and recompile our example with a version of Java that supports Loom, then we can execute the resulting code. This time, the program runs to completion without an issue. This is a good example of the Loom philosophy in action—localizing the change applications need to make to just the code locations that create threads.

One way in which the new thread library encourages developers to move on from older paradigms is that subclasses of `Thread` cannot be virtual. Therefore, code that subclasses `Thread` will continue to be created using traditional OS threads.

NOTE Over time, as virtual threads become more common and developers stop caring about the difference between virtual and OS threads, this should discourage the use of the subclassing mechanism because it will always create an OS-schedulable thread.

The intention is to protect existing code that uses subclasses of `Thread` and follow the principle of least surprise.

Various other parts of the thread library also need to be upgraded to better support Loom. For example `ThreadBuilder` can also build `ThreadFactory` instances that can be passed to various `Executors` like this:

```
jshell> var tb = Thread.ofVirtual();
tb ==> java.lang.ThreadBuilders$VirtualThreadBuilder@312b1dae

jshell> var tf = tb.factory();
tf ==> java.lang.ThreadBuilders$VirtualThreadFactory@506e1b77

jshell> var tb = Thread.ofPlatform();
```

```
tb ==> java.lang.ThreadBuilders$PlatformThreadBuilder@1ddc4ec2

jshell> var tf = tb.factory();
tf ==> java.lang.ThreadBuilders$PlatformThreadFactory@b1bc7ed
```

Virtual threads will need to be attached to an actual OS thread to execute. These OS threads upon which a virtual thread executes are called *carrier threads*. We have already seen carrier threads in some `jshell` output in one of our earlier examples. However, over its lifetime, a single virtual thread may run on several different carrier threads. This is somewhat reminiscent of the way that regular threads will execute on different physical CPU cores over time—both are examples of execution scheduling.

3.3.3 Programming with virtual threads

The arrival of virtual threads brings with it a change of mindset. Programmers who have written concurrent applications in Java as it exists today are used to having to deal (consciously or unconsciously) with the inherent scaling limitations of threads.

We are used to creating task objects, often based on `Runnable` or `Callable` and handing them off to executors, backed by thread pools, which exist to conserve our precious thread resources. What if all of that was suddenly different?

In essence, Project Loom tries to solve the scaling limitation of threads by introducing a new concept of a thread that is cheaper than existing notions and that doesn't directly map to an OS thread. However, this new capability still looks and behaves like a thread as Java programmers already understand it.

Rather than requiring developers to learn a completely new programming style (such as continuation-passing style or the Promise/Future approach or callbacks), the Loom runtime keeps the same programming model we know from today's threads for virtual threads; virtual threads are threads, at least as far as the programmer is concerned.

Virtual threads are *preemptive* because user code does not need to explicitly yield. Scheduling points are up to the virtual scheduler and the JDK. Users must make no assumptions on when they happen, because this is purely an implementation detail. However, it is worth understanding the basics of operating system theory that underlies scheduling to appreciate how virtual threads differ.

When the operating system schedules platform threads, it allocates a *timeslice* of CPU time to a thread. When the timeslice is up, a hardware interrupt is generated, and the

kernel is able to resume control, remove the executing platform (user) thread, and replace it with another.

NOTE This mechanism is how Unix (and assorted other operating systems) has been able to implement time-sharing of the processor among different tasks—even decades ago in the era when computers had only one processing core.

Virtual threads, however, are handled differently from platform threads. None of the existing schedulers for virtual threads uses timeslices to preempt virtual threads.

NOTE Using timeslices for preemption of virtual threads would be possible, and the VM is already capable of taking control of executing Java threads—it does so at JVM safepoints, for example.

Instead, virtual threads automatically give up (or *yield*) their carrier thread when a blocking call (such as I/O) is made. This is handled by the library and runtime and is not under the explicit control of the programmer.

Thus, rather than forcing programmers to explicitly manage yielding, or relying upon the complexities of nonblocking or callback-based operations, Loom allows Java programmers to write code in traditional, thread-sequential style. This has additional benefits such as allowing debuggers and profilers to work in the usual way. Toolmakers and runtime engineers need to do a bit of extra work to support virtual threads, but that's better than forcing an additional cognitive burden onto end user Java developers. In particular, this approach differs from the `async` / `await` approach adopted by some other programming languages.

The designers of Loom expect that, because virtual threads need never be pooled, they *should* never be pooled, and instead the model is the unconstrained creation of virtual threads. For this purpose, an *unbounded executor* has been added. It can be accessed via a new factory method, `Executors.newVirtualThreadPerTaskExecutor()`. The default scheduler for virtual threads is the work-stealing scheduler introduced in `ForkJoinPool`.

NOTE It is interesting how the work-stealing aspect of Fork/Join has become far more important than the recursive decomposition of tasks.

The design of Loom as it is today is predicated on the developer understanding the computational overhead that will be present on the different threads in their application. Simply put, if a vast number of threads all need a lot of CPU time constantly, your application has a resource crunch that clever scheduling can't help. On the other hand, if only

a few threads are expected to become CPU-bound, these should be placed into a separate pool and provisioned with platform threads.

Virtual threads are also intended to work well in the case where there are many threads that are CPU-bound only occasionally. The intent is that the work-stealing scheduler will smooth out the CPU utilization and real-world code will eventually call an operation that passes a yield point (such as blocking I/O).

3.3.4 When will Project Loom arrive?

Loom development is taking place in a separate repo, not on the JDK mainline. Early-access binaries are available, but these still have some rough edges—crashes still occur but are becoming less common. The basic API is taking shape, but it is almost certainly not completely finalized yet.

JEP 425 (**https://openjdk.java.net/jeps/425**) has been filed to integrate virtual threads as a Preview feature, but at the time of writing, this JEP has not been targeted to any release yet. It is reasonable to suppose that if it is not included as Preview in Java 19, then a final version of the feature will not be available as part of Java 21 (which is likely to be the next LTS version of Java). There is still a lot of work to be done on the APIs that are being built on top of virtual threads, such as structured concurrency and other more advanced features.

One key question that developers always have is about performance, but this is always difficult to answer during the early stages of development of a new technology. For Loom, we are not yet at the point where meaningful comparisons can be made and the current performance is not thought to be really indicative of the final version.

As with other long-range projects within OpenJDK, the real answer is that it will be ready when it's ready. For now, there is enough of a prototype to start experimenting with it and get a first taste of what future development in Java might look like. Let's turn our attention to the last of the four major OpenJDK projects that we're discussing: Valhalla.

18.4 Project Valhalla

To align JVM memory layout behavior with the cost model of modern hardware.

—Brian Goetz

To understand where the current Java model of memory layout reaches its limits and starts to break down, let's start with an example. In figure 18.1, we can see an array of primitive ints. Because these values are primitive types and not objects, they are laid out at adjacent memory locations.

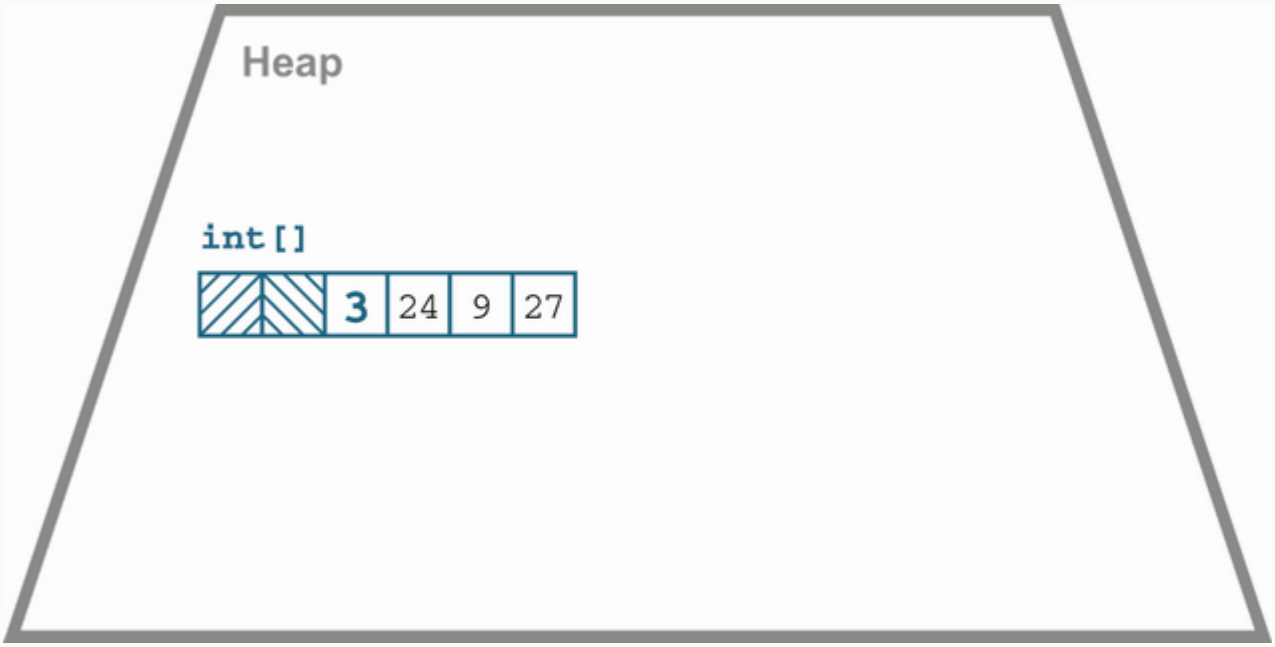


Figure 18.1 Array of primitive ints

To see the difference with object arrays, let's contrast this with the boxed integer case. An array of `Integer` objects will be an array of references, as shown in figure 18.2.

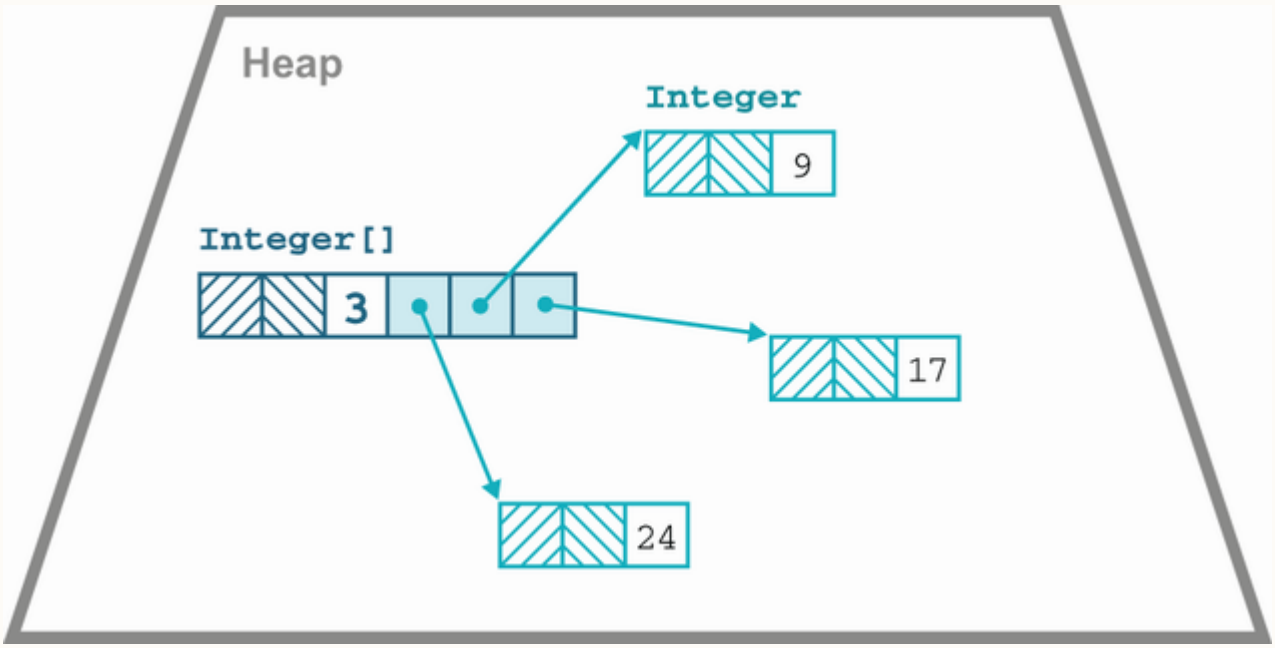


Figure 18.2 Array of `Integer` objects

Because each `Integer` is an object, it is required to have an object header, as we explained in the previous chapter. We sometimes say that each object is required to pay the

"header tax" that comes with being a Java object.

For over 20 years, this memory layout pattern has been the way that the Java platform has functioned. It has the advantage of simplicity but has a performance trade-off: dealing with arrays of objects involves unavoidable pointer indirections and attendant cache misses.

As an example, consider a class that represents a point in three-dimensional space, a `Point3D` type. It really comprises only three spatial coordinates and, as of Java 17, can be represented as an object type with three fields (or an equivalent record) like this:

```
public final class Point3D {
    private final double x;
    private final double y;
    private final double z;

    public Point3D(double a, double b, double c) {
        x = a;
        y = b;
        c = z;
    }

    // Additional methods, e.g getters, toString() etc.
}
```

In HotSpot, an array of these point objects is laid out in memory, as shown in Figure 18.3.

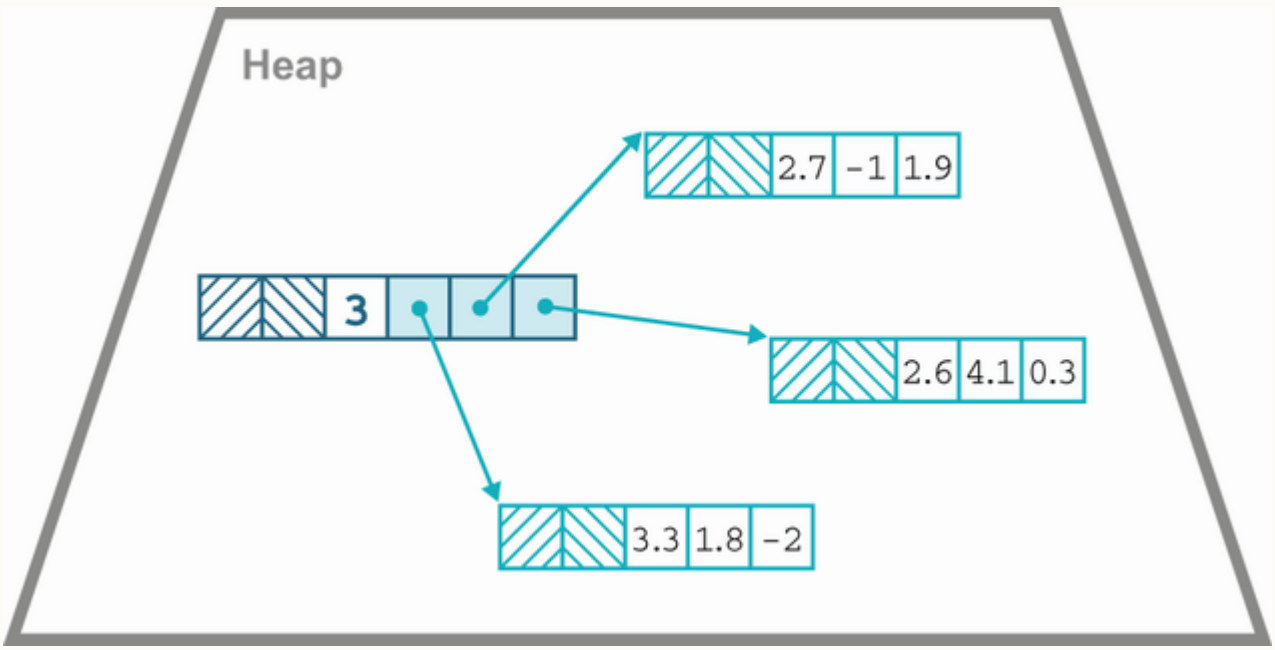


Figure 18.3 An array of `Point3D`

When processing this array, each element is accessed via an additional indirection to get the coordinates of each point. This causes a cache miss for each point in the array, degrading performance for no good reason.

For programmers who care a lot about performance, the ability to define types that can be laid out in memory more effectively would be very useful. We should also note that object identity has no real benefit for the programmer when working with `Point3D` values, because two points should be equal if and only if all their fields are equal.

This example demonstrates the following two separate programming concepts that are both enabled by the removal of object identity:

- *Heap flattening*—The removal of pointer indirection for identity-less objects resulting in higher memory density
- *Scalarization*—The ability to break up an identity-less object into fields and reconstitute it again elsewhere if needed

These separate properties will turn out to have consequences for the user model for identity-less objects.

NOTE It turns out that scalarization—the ability of the VM to break up and reconstitute value objects as much as it likes—is surprisingly useful. The JVM contains a JIT technique called *escape analysis* that can greatly benefit from the freedom to split a value object into its individual fields and flow them through the code separately.

Keeping these properties in mind, we can also approach Valhalla starting from the question: "can we avoid paying the header tax?" Broadly, the answer is yes, provided the following:

- The objects don't need a concept of identity.
- The class is final, so all targets for method calls can be known at class loading time.

Basically, the first property removes the need for the mark word of the header, and the second greatly reduces the need for the klass word (see chapters 4 and 17 for more on the klass word).

The klass word is still required while the object is on the heap, unless it has been flattened as an instance field in another object or flattened as an element of array because the object field layout might need to be described—for example, so that the GC can walk the object graph. However, when the objects have been scalarized, we can drop the header.

From a developer's perspective, therefore, one of the main outcomes of Valhalla is the arrival of a new form of values in the Java ecosystem, referred to as *value objects*, which are instances of *value classes*. These new types are understood to be (usually) small, immutable, identity-less types.

NOTE Value classes have been referred to by several different names during their development, including *primitive classes* and *inline types*. Naming things is hard, especially in a mature language that may well have used many of the common names for language concepts.

Example use cases for value classes include

- New varieties of numerics, such as unsigned bytes, 128-bit integers, and half-precision floats
- Complex numbers, colors, vectors, and other multidimensional numerical values
- Numbers with units: sizes, temperatures, velocity, cashflow, and so on
- Map entries, database rows, and types for multiple-return
- Immutable cursors, subarrays, intermediate streams, and other data structure view abstractions

There is also the possibility that some existing types could be retrofitted and evolve to become represented as value classes. For example, `Optional` and much of `java.time` are obvious candidates that could become value classes in a future release if it proves to be feasible.

NOTE Records are not per se related to value classes, but it is highly likely that a number of Records will be aggregates that do not require identity, so the concept of a *value record* may be a very useful one.

If this new form of value can be implemented on the JVM, then for classes such as the spatial points we've been discussing, a flattened memory layout such as that shown in figure 18.4 would be far more efficient.

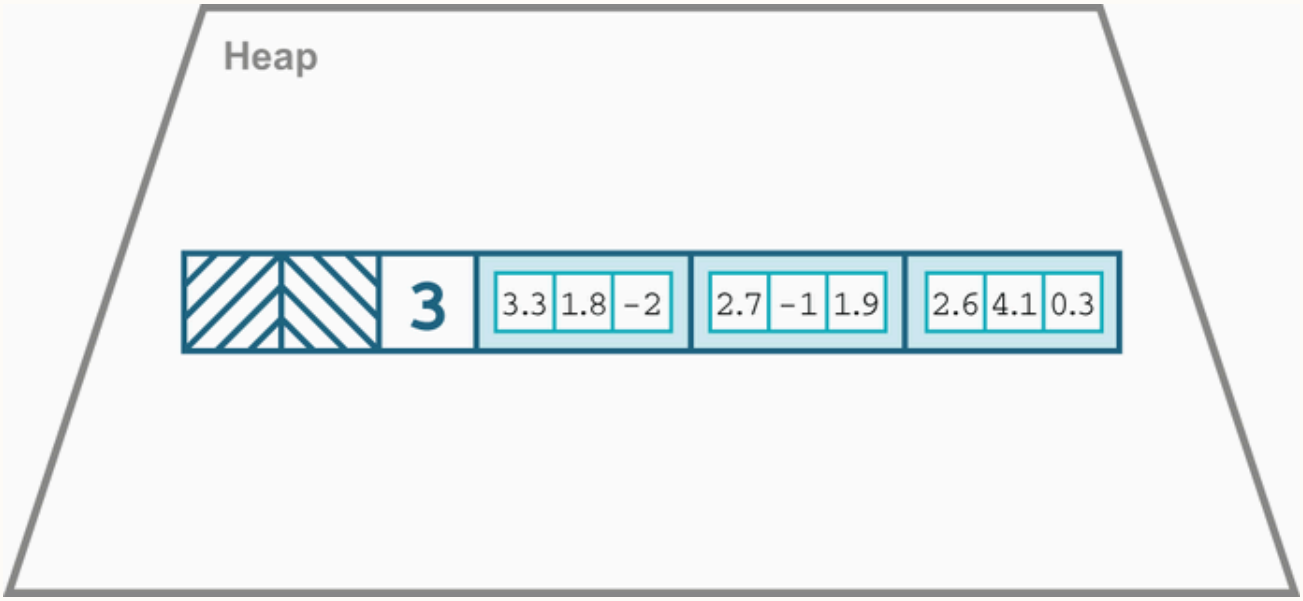


Figure 18.4 Array of inline points

This memory layout would be something close to what a C programmer would recognize as an array of `struct`, but without exposing the full dangers of low-level memory access. The flattened layout reduces not only memory footprint but also the load on garbage collectors.

18.4.1 Changing the language model

The biggest change that needs to be made is to modify the concept of `java.lang.Object` as a universal superclass because it has methods on it such as `wait()` and `notify()` that are inherently linked to object identity. Without the object header, there is no mark word to store the object's monitor and nothing to wait upon. The object does not really have a well-defined lifetime either because it can be freely copied and the resulting copies are indistinguishable. Instead, two new interfaces are defined in `java.lang`: `IdentityObject` and `ValueObject`. JEP 401 (see **https://openjdk.org/jeps/401**) describes value objects in detail, but basically all value classes implicitly implement `ValueObject`.

All identity classes will implicitly implement `IdentityObject`, and all preexisting concrete classes are opted-in as identity classes. Existing interfaces and (most) abstract classes do not extend either of the new interfaces. API designers may wish to update their interfaces to explicitly extend `IdentityObject` if their capabilities are not compatible with the new semantics.

Value classes are `final` and may not be `abstract`. They may not implement (directly or indirectly) `IdentityObject`. An `instanceof` test can be used to check whether or not an object is a value object.

NOTE As well as not being able to `wait()` or `notify()` on value objects, it will not be possible to have synchronized methods or blocks because value objects do not have a monitor.

The class `Object` itself will undergo some subtle repositioning because it will not implement either `IdentityObject` or `ValueObject` and will become more similar to an abstract class or interface. Code such as this

```
var o = new Object();
```

will also change meaning—it is anticipated that `o` will contain an instance of some anonymous subclass of `Object`, which for backward-compatibility reasons will be understood to be an identity class.

Although the initial aims of value classes seem clear, it turns out to have some far-reaching consequences. For the project to be successful, it is necessary to consider the logical conclusions of introducing a third form of value.

3.4.2 Consequences of value objects

Assignment of value objects has fairly obvious semantics: the bits are copied, just as they are for primitives (assignment of references also copies the bits, but in that case, we end up with two references to the same heap location). However, what happens if we need to construct a value object that is not completely identical to the original but is a modified copy?

Recall that value objects are immutable—they have only `final` fields. This means that their state cannot be changed by `putfield` operations. Instead, another mechanism is needed to produce a value object that has a different state than the original. To achieve this, some new bytecodes will be needed, shown next:

- `aconst_init`
- `withfield`

The new `withfield` instruction essentially acts as a bytecode-level equivalent to the use of wither methods (which we discussed in chapter 15).

The other new instruction, `aconst_init`, provides a default value for an instance of a value class. Let's take a closer look at why this is needed.

In Java to date, both primitives and object references are understood to have a default value that corresponds to "all bits zero," with `null` being understood to be the meaning of zero bits for references. However, when we try to extend these semantics to handle value objects, we find that there are two related issues:

- Some value classes do not have a good choice for default value.
- The possibility of *value tearing*.

The no-good-default issue really comes down to wanting to be able to say that the value object isn't really a value yet, but Java already has a way to do that: `null`. Furthermore, users are already used to dealing with `null` point exceptions (NPEs) when they have an uninitialized value.

The second problem, tearing, is really an old problem in a new guise. In older versions of Java, running on 32-bit hardware had some subtle potential problems when handling 64-bit values (such as longs). In particular, writes to longs were performed as two separate (32-bit) writes, and it was possible for a reading thread to observe a state where only one of the 32-bit writes had completed. This would allow the reading thread to observe a "torn" value for the long—one that was neither the before nor after state.

Value types have the possibility to reintroduce this issue. If values can be scalarized, then how can we guarantee the atomicity of writes?

The solution is to recognize that value classes represent not one new form of data but *two*. If we want to avoid tearing, we need to use a reference. This is the well-established idea that using a layer of indirection allows us to update values without tearing them.

Also, consider that some classes have no sensible default that could correspond to zero bits. For example, what would be the default of `LocalDate` after it has migrated to a value class? Some might argue that zero bits should be interpreted as zero offset from epoch (i.e., 1st Jan 1970), but this seems deeply error prone.

This bring us to the concept of *identity-free references*—basically, objects we have removed identity from. At a low level, this still allows calling convention optimizations in the JIT (e.g., scalarized passing of values) and scalarization in JIT code but gives up the memory improvements on the heap. These objects are always handled by reference, just as identity objects are, and they have a straightforward default of `null`. Setting a non-trivial default value for the object is then handled by the constructors or factory methods, just as it should be.

In addition, for advanced use cases, there are also *primitive value types*. These act more like the built-in "true primitives" and allow flattening in the heap as well as the scalarization allowed with value objects. The additional benefits come with associated costs, though—the requirement of accepting zero bits as the default value as well as the possibility of tearing under updates that might have data races.

NOTE The intent is that primitive value types are really only meant for small values (64–128 bits or less on today's hardware) and will need additional care when programming with them.

Tearing does expose users to potential security problems, although it is tempting to say it is an issue only for "bad" programs that have data races. It is, in any case, a new form of possible concurrency bug and will require locking to properly defend against.

One other aspect of primitive value classes is that the runtime needs to know how to lay them out in memory. For this reason, it is not possible to create a field of a primitive value class that refers to (either directly or indirectly) the declaring class. In other words, instances of primitive value classes cannot contain a cyclic data structure of primitive value classes—they must have fixed-size layouts, so that they can be flattened in the heap. Overall, the expectation is that most users will want to use identity-free value objects and that the extended primitives will be used much more rarely.

To conclude this section, let's look at one other aspect of how value classes are represented in bytecode. Recall that way back in chapter 4, we met the concept of a type descriptor. Identity reference types are denoted in bytecode via *L-type descriptors* such as `Ljava/lang/String;` for a string.

To describe values of primitive classes, a new basic type is being added, the *Q-type descriptor*. A descriptor beginning with `Q` has the same structure as an L-descriptor (e.g., `QPoint3D;` for a primitive class `Point3D`). Both Q and L values are operated on by the same set of bytecodes, that is, those that start with `a`, such as `aload` or `astore`.

The values referred to via Q-descriptors (sometimes called "bare" objects) have the following major differences from references to value objects:

- References to value objects, like all object references, can be `null`, whereas bare values cannot.
- Loads and stores of references are atomic with respect to each other, whereas loads and stores of sufficiently large bare values may tear, as is the case for long and double on a 32-bit implementation.

- Object graphs may not be circular if linked via a path of Q-descriptors: a class `C` cannot reference `QC;` in its layout, either directly or indirectly.
- For technical reasons, the JVM is required to load classes named in Q-descriptors much earlier than those named by L-descriptors.

These properties are basically the bytecode-level encoding of some of the properties of value and primitive objects that we have already met.

There is one final piece of the puzzle that we should briefly consider before moving on from Valhalla: a need to revisit the subject of generic types. This arises quite naturally as a consequence of the introduction of value and primitive objects.

3.4.3 Generics revisited

If Java is to include value classes, the question naturally arises as to whether value classes can be used in generic types, for instance, as the value for a type parameter. If not, this would seem to greatly limit the usefulness of the feature. Therefore, the high-level design has always included the assumption that value classes will eventually be valid as values of type parameters in an enhanced form of generics.

Fortunately, the role of `Object` has subtly changed in Valhalla—it has been retrospectively altered to be the superclass of both value and identity objects. This allows us to include value objects within the realm of existing generics. However, the integration of primitive types into this model is also desirable.

The long-term intent is to be able to extend generics—to allow abstraction over all types, including value classes and the existing primitives (and `void`). If this project is to be successful, we need to be able to compatibly evolve existing libraries—especially the JDK libraries—to fully take advantage of these features.

Partially, this work would also involve updating the basic value types (`int`, `boolean`, etc.) to become primitive value classes, so that basic primitive values become primitive objects. This would also mean that the wrapper classes would be repurposed to fit into the primitive classes model.

This extension to generics will produce a form of *generic specialization* for the primitives. This brings in aspects of a generic programming system similar to that found in other languages, such as templates in C++. At time of writing, the generics work is still at an early stage, and all JEPs pertaining to it are still in Draft state.

18.5 Java 18

It is the nature of any text that attempts to be forward-looking that it will inevitably be out of date by the time it is read. At the time of writing, Java 17 has been delivered, and Java 18 was delivered in March 2022. The following JEPs were targeted at Java 18 and formed the content of the new release:

- JEP 400 UTF-8 by Default
- JEP 408 Simple Web Server
- JEP 413 Code Snippets in Java API Documentation
- JEP 416 Reimplement Core Reflection with Method Handles
- JEP 417 Vector API (Third Incubator)
- JEP 418 Internet-Address Resolution SPI
- JEP 419 Foreign Function and Memory API (Second Incubator)
- JEP 420 Pattern Matching for `switch` (Second Preview)
- JEP 421 Deprecate Finalization for Removal

Of these, the UTF-8 changes, the changes to Core Reflection, and the deprecation of finalization are internal changes that provide some tidying up and simplification of the internals that can be built on in future releases.

The Vector and Foreign API updates are the next milestone on the journey toward Panama, and the next iteration of Pattern Matching is the next step for Amber. Java 18 does not contain any JEPs that deliver any part of Loom or Valhalla. Nothing has been confirmed at time of writing, but it is rumored that the first version of Loom will be delivered as a Preview feature in Java 19 (expected in September 2022).

Summary

One of Java's original design principles was that the language should evolve carefully—that a language feature should not be implemented until the wider impact on the language as a whole was fully understood. Other languages can, and do, move faster than Java, which occasionally leads to complaints from developers that "Java needs to evolve faster." However, the flip side of this is that other languages may "advance at speed and repent at leisure." A flawed design, once integrated into the language, is essentially there forever.

Java's approach, on the other hand, is to proceed conservatively—to make sure that a feature is understood, including all of its consequences, before committing to it. Let

other languages break new ground (or, the cynic might say, be first "over the top") and then see what conclusions can be drawn from their experimentation.

In fact, this sort of influence—the back-and-forth borrowing of language concepts—is a common feature of language design. It also provides a great example of the idea sometimes expressed as "Great artists steal." This quote is often attributed to Steve Jobs, but he did not invent it—he had merely borrowed (or stolen) it from other thinkers.

In actuality, this idea seems to have been invented multiple times, but one of the original forms of it that can be definitively traced is this one:

One of the surest of tests is the way in which a poet borrows. Immature poets imitate; mature poets steal; bad poets deface what they take, and good poets make it into something better, or at least something different.

—T. S. Eliot

Eliot's point applies as readily to language designers as it does to poets. Truly great programming languages (and language designers) borrow (or steal) from each other freely. Good ideas that are first expressed in one language do not remain solely confined there—in fact, that is one of the ways that we know the idea was good in the first place.

In this final chapter, we have met the four major ongoing projects within OpenJDK. Taken together, they aim to deliver a radically different version of future Java. Some of these projects are very ambitious, others more modest. They will all be delivered as part of the regular cadence of Java releases. The Java we write in a year, or three, from now may well look quite unlike what we would write today.

The major aspects that we might guess will be reshaped include

- *The merger of object and functional programming*—Amber introduces new language features that converge these models.
- *Threading*—Loom will introduce a new model for threads that engage in I/O.
- *Memory layout*—Valhalla solves several problems at once, improving memory density as well as extending generics.
- *Better native interoperability*—Panama helps undo some of the design problems with JNI and other native technologies.
- *Ongoing cleanup of internals*—A series of JEPs to slowly remove aspects of the platform that are no longer needed.

The ultimate shape of future Java is still to be determined—the future is unwritten as of now. What is sure, however, is that after more than 25 years, Java is still a force to be reckoned with. It has already survived several major transitions in the world of software—a track record to be proud of, and one that bodes well for the future.

Previous chapter
< 17 Modern internals

Next chapter
Appendix A. Selecting your Java >